

Recursividad

...

Programación
2025

Cálculo del factorial

- El factorial de un entero no negativo n (escrito $n!$ y pronunciado n factorial) se calcula como el producto:

$$n! = n * (n - 1) * (n - 2) * \dots * 1$$

- Por ejemplo:

$$4! = 4 * 3 * 2 * 1$$

$$5! = 5 * 4 * 3 * 2 * 1 = 5 * 4!$$

- Por la tanto la definición puede escribirse como:

$$n! = n * (n - 1)!$$

$$4! = 4 * 3!$$



$$3! = 3 * 2!$$



$$2! = 2 * 1!$$



$$1! = 1$$

Funciones dentro de funciones

- Al definir una función, es posible incorporar sentencias que llamen a otras funciones.
- La función original no continúa hasta no obtener los resultados de la función a la que llama.

Funciones dentro de funciones

- Ejemplo: cálculo del desvío estándar de una lista de valores.

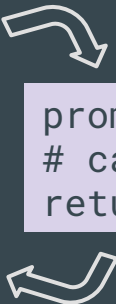
```
array = [1, 3, 5, 7]  
desv = desvio(array)  
print(desv)
```

Funciones dentro de funciones

- Ejemplo: cálculo del desvío estándar de una lista de valores.

```
array = [1, 3, 5, 7]  
desv = desvio(array)
```

```
print(desv)
```



```
prom = media(array)  
# calculos ...  
return sd
```

Funciones dentro de funciones

- Ejemplo: cálculo del desvío estándar de una lista de valores.

```
array = [1, 3, 5, 7]  
desv = desvio(array)
```

```
print(desv)
```

Programa principal

```
prom = media(array)
```

```
# calculos ...  
return sd
```

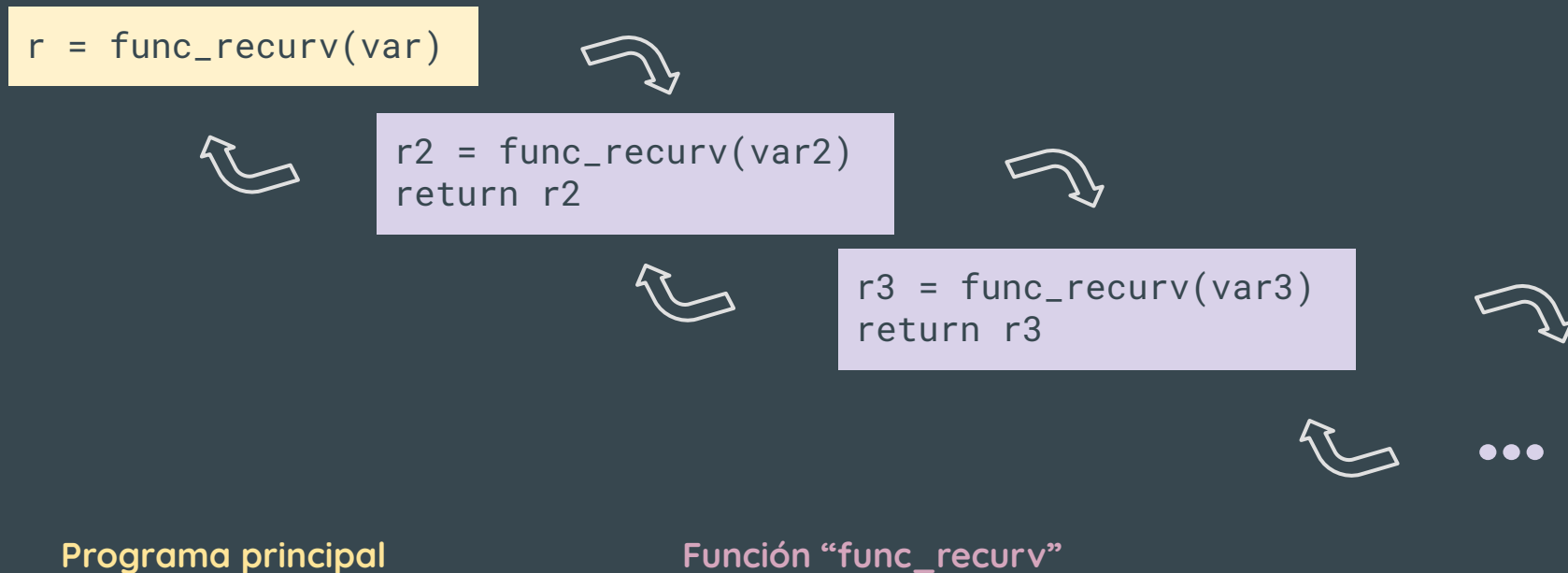
Función “desvio”

```
for i in array:  
    # mas calculos...  
return promedio
```

Función “media”

Recursividad

- Una función **recursiva** es aquella que se llama a sí misma.



Componentes de la recursividad

- Todo algoritmo recursivo tiene que tener, al menos, 2 componentes:
 - Componente recursivo.
 - Componente base.
- El **componente** o **caso base** es un caso que tiene una solución directa (no recursiva).

$$f(n) \begin{cases} 1 & n \leq 1 \\ n * f(n - 1) & n > 1 \end{cases}$$

Implementando función “factorial”

```
def factorial_recursivo (n):  
    if n <= 1:          # Caso base  
        return 1  
    else:               # Caso recursivo  
        resultado = n * factorial_recursivo(n-1)  
        return resultado
```

```
fact_rec = factorial_recursivo(10)  
print("Recursivo: ", fact_rec)
```

```
Recursivo: 3628800
```

Recursión vs Iteración

- Por definición, todo problema que pueda resolverse mediante recursión se puede resolver por iteración.
- Las soluciones recursivas suelen ser más lentas y demandar más memoria, pero suelen ser más fáciles de comprender, implementar y depurar.
- Si una solución de un problema se puede expresar iterativa o recursivamente con igual facilidad, **es preferible la solución iterativa.**

Implementando función “factorial”

```
def factorial_recursivo (n):  
    if n <= 1:          # Caso base  
        return 1  
    else:               # Caso recursivo  
        resultado = n * factorial_recursivo(n-1)  
        return resultado
```

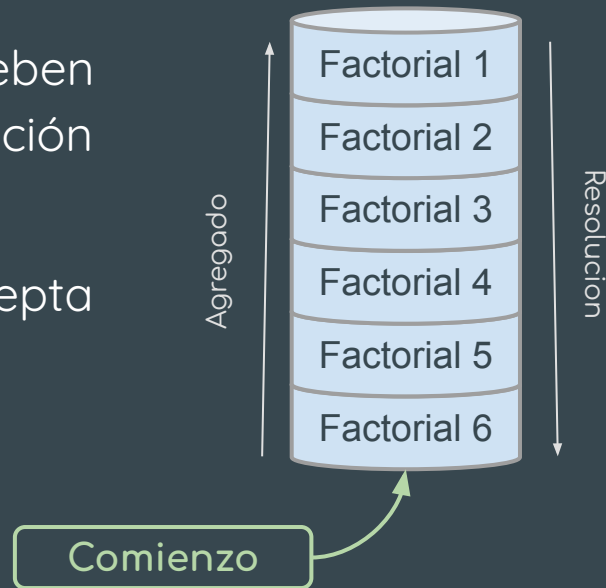
```
def factorial_iterativo (n):  
    resultado = 1  
    for i in range (2,n+1):  
        resultado *= i  
    return resultado
```

```
fact_rec = factorial_recursivo(10)  
fact_iter = factorial_iterativo(10)  
print("Recursivo: ", fact_rec, "Iterativo", fact_iter)
```

```
Recursivo: 3628800 Iterativo 3628800
```

Límite de recursividad

- Por cada llamada dentro de una función recursiva, se agrega un valor a una **pila de recursión**.
- Esta pila mantiene el orden en que se deben resolver las distintas llamadas a la función recursiva.
- Por defecto, en Python, esta cola acepta acepta 1000 valores (luego da error).



Precauciones!!

- Siempre debe haber, al menos, **un caso base**; sino la llamada recursiva se hace **infinita**.
- Si la llamada anterior requiere un resultado de una siguiente, siempre debe haber un **return**.
- Cuando se trabaja con tipos de datos no mutables (listas), no hace falta retornarlos: todos los cambios se mantienen!

Ejemplo

- Implemente la operación “multiplicación” como una suma recursiva.

Ejemplo: pseudocódigo

- Implemente la operación “multiplicación” como una suma recursiva.

Pseudocódigo:

Recibir dos números: multiplicando y multiplicador

si multiplicador == 1:

 devolver multiplicando

sino:

 calcular multiplicando + f(multiplicando, multiplicador-1)

Ejemplo: código

```
def mult (multiplicando, multiplicador):  
    print("multiplicando: ", multiplicando) # Solo para visualizar la recursión  
    print("multiplicador: ", multiplicador)  
    if multiplicador == 1:  
        return multiplicando  
    else:  
        return multiplicando + mult(multiplicando, multiplicador-1)  
  
resultado = mult(4,6)  
print(resultado)
```


Ejemplo

```
(base) matias@rfgenom002:~$ python test.py
multiplicando: 4
multiplicador: 6

multiplicando: 4
multiplicador: 5

multiplicando: 4
multiplicador: 4

multiplicando: 4
multiplicador: 3

multiplicando: 4
multiplicador: 2

multiplicando: 4
multiplicador: 1

24
```